School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00172134
Student Name:	Andrei Scarevnea

Module Information

Module Title:	Fundamentals of AI & Data Science
Module Code:	COMP H270

Assessment Information

Assessment Title:	Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy. https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☑ I have completed the TU Dublin AI Driving License theory test. https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Scarevnea
Date:	27.04.2026

Contents

Executive Summary 3
Introduction 3
Business Understanding 4
Cybersecurity Problem 4
Business Objective 4
Success Criteria 4
Data Understanding & Exploration 5
Dataset Overview 5
Descriptive Statistics 5
Class Distribution and Imbalance 6
Data Quality Assessment 7
Visualisation of Histogram - Analysis 8
Box Plot Visualisation\Analys 9
Visualisation of Heatmap - Correlation Analysis 10
Methodology/Data Preparation 11
Data Selection — Removing Useless Attributes 11
Data Transformation - Inconsistent Categorical Values 11
Handling Missing Values 12
Feature Engineering — Creating New Attributes 12
Encoding Categorical Features 13
Train/Test Split 13
Feature Scaling 14
Modelling 14
Classification — Complement Naïve Bayes 14
Classification — Decision Tree 16
Regression — Linear Regression and Decision Tree Regressor 17
Evaluation & Results 18
Classification Model Comparison 18
Feature Importance 18
Regression Model Comparison 19
Conclusions and Reflection on Learning 20
Summary of Findings 20
Reflection on Learning 20
References 21

Executive Summary

This project presents application of machine learning classification techniques to analyse a cybersecurity related dataset representing network activity patterns within a financial organisation. The dataset is made up of 6000 session records across 19 features that will be used to identify malicious behaviour and support early detection of cyberattacks.

Following the CRISP-DM framework, the dataset was explored, prepared, modelled, and evaluated using Naïve Bayes and Decision Tree classifiers to predict whether a network session is an Attack or Benign. Two regression models were built to predict session_duration: Linear Regression and a Decision Tree Regressor.

The main finding is that anomaly_score is one of the most important features to rank if is an attack or not, accounting for 81.2% of the Decision Tree's feature importance. The tuned Decision Tree and tuned Naive Bayes both achieved 75% overall accuracy, with attack recall of 98% — meaning the models successfully detect nearly all attacks. Linear Regression outperformed the Decision Tree Regressor on the regression task (R^2 = 0.48 vs 0.37), suggesting session_duration has a roughly linear relationship with the available features.

Introduction

Financial organisations are always in hunted by threat actors due to high value of their digital assets and users' data. Detecting malicious network activity can reduce operational disruption and data lose. Machine learning classifiers can provide an effective method for identifying suspicious network traffic and detect attacks before they cause damage.

In this project, using CRISP-DN framework will help us to develop a predictive model capable to analyse a cybersecurity dataset and develop predictive classification models capable of differentiate normal and possibly malicious activity in large datasets. The objective is not only to build accurate predictive models but also to interpret the dataset and extract important understandings that support cybersecurity decision-making.

Business Understanding

Cybersecurity Problem

A financial organisation has experienced several cyberattacks targeting its network infrastructure, costing millions in losses. As defend measurements, the organisation requires an automated system able to recognise malicious network sessions from legitimate ones in real-time, allowing security analysts to take actions before damage occurs.

Business Objective

The primary business objective is to build a reliable model for detecting attack sessions in network traffic logs.

A secondary objective is to understand session duration behaviour, which can help with network capacity planning and anomaly threshold setting.

Applying the mapping from business objectives to data mining tasks described in the CRISP-DM framework (Week 2 lectures), the business objectives reflect to the following data mining goals:

- Classification: Predict is_attack (Attack / Benign) from network session features.
- Regression: Predict session_duration as a continuous numerical value.

Success Criteria

In a cybersecurity context, a missed attack (false negative) is more costly and harmful than a false alarm (false positive). A model that labels every session as Benign would achieve 74.7% accuracy because the dataset is imbalanced yet, so it would cause a real problem in recognising an attack. For this reason, the primary success action is recall for the Attack class. A model that achieves high recall (>= 0.85) for attacks while maintaining reasonable overall accuracy is considered successful. This aligns with the CRISP-DM requirement to define success criteria in business terms before modelling begins.

Data Understanding & Exploration

Dataset Overview

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 19 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   session_id        6000 non-null   object
 1   protocol          5580 non-null   object
 2   device_type       6000 non-null   object
 3   vpn_status        6000 non-null   object
 4   region            6000 non-null   object
 5   login_attempts    6000 non-null   int64
 6   failed_logins     6000 non-null   int64
 7   bytes_sent        6000 non-null   float64
 8   bytes_received    5580 non-null   float64
 9   packet_rate       6000 non-null   float64
 10  connection_count  6000 non-null   int64
 11  tcp_flags         6000 non-null   object
 12  port_number       6000 non-null   int64
 13  session_retries   6000 non-null   int64
 14  signal_strength   5580 non-null   float64
 15  privilege_level   6000 non-null   object
 16  anomaly_score     6000 non-null   float64
 17  session_duration  6000 non-null   float64
 18  is_attack         6000 non-null   object
dtypes: float64(6), int64(5), object(8)
memory usage: 890.8+ KB
```

The dataset contains 6,000 instances and 19 features. Each row represents a single network session. The dataset has a mix of numerical and categorical features, with one nominal identifier (session_id), six categorical attributes, and the rest of them numerical features. There are two potential target variables: is_attack (categorical variable, for classification) and session_duration (numerical – ratio variable, for regression).

Using the terminology from Week 2 lectures, session_id is a nominal variable (unique identifier with no predictive power), protocol/device_type/vpn_status/region/tcp_flags/privilege_level are categorical variables (named groups with no inherent order), and the rest of all numerical features are ratio variables (they have a true zero and support meaningful arithmetic operations).

Descriptive Statistics

```
# — Descriptive Statistics —
df[['failed_logins', 'anomaly_score', 'session_duration', 'bytes_sent', 'bytes_received']].describe()
```

	failed_logins	anomaly_score	session_duration	bytes_sent	bytes_received
count	6000.000000	6000.000000	6000.000000	6000.000000	5580.000000
mean	1.020667	0.283358	92.914416	5275.753584	5981.881281
std	1.004527	0.159846	16.595032	3132.622483	1994.673822
min	0.000000	0.002210	32.736351	100.000000	100.000000
25%	0.000000	0.157360	81.428235	4009.308961	4662.928544
50%	1.000000	0.260576	92.878546	5013.713497	5992.395457
75%	2.000000	0.389571	104.194079	6060.269321	7333.109710
max	7.000000	0.824099	156.431365	48743.941649	13019.206996

The most relevant key numerical features that deliver important patterns are 'failed_logins', 'anomaly_score', 'session_duration', 'bytes_sent', 'bytes_received':

- failed_logins has a mean of 1.02 and a maximum of 7. The distribution is discrete and right-skewed.
- anomaly_score ranges from 0.002 to 0.824 with a mean of 0.283 and standard deviation of 0.160. The distribution is right-skewed, meaning most sessions have low anomaly scores with a tail of higher values.
- session_duration has a mean of 92.9 seconds and a standard deviation of 16.6, suggesting an approximately normal distribution - an important pattern for regression.
- bytes_sent and bytes_received are continuous and show considerable spread, suggesting some sessions transfer much more data than others.

```
# — Descriptive Statistics —
for col in ['protocol', 'vpn_status', 'is_attack']:
    print(col)
    print(df[col].value_counts(dropna=False))
    print("\n")
```

```
protocol
protocol
TCP     2532
UDP     1378
tcp      572
Udp      557
ICMP     541
NaN      420
Name: count, dtype: int64


vpn_status
vpn_status
Off    4184
On     1816
Name: count, dtype: int64


is_attack
is_attack
Benign    4481
Attack    1519
Name: count, dtype: int64
```

```
# — Descriptive Statistics —
df[['protocol', 'vpn_status', 'is_attack']].describe()
```

	protocol	vpn_status	is_attack
count	5580	6000	6000
unique	5	2	2
top	TCP	Off	Benign
freq	2532	4184	4481

For categorical features, protocol has the following values (TCP, UDP, ICMP) with most frequent TCP protocol, while vpn_status is binary (On / Off) with a frequency of 2532 for Off status. The class label is_attack contains 4,481 Benign and 1,519 Attack records indicating that most of the traffic is not an Attack.

Class Distribution and Imbalance

```
# — Class Balance —
print('is_attack distribution:\n')

print(df['is_attack'].value_counts(normalize=True).mul(100).round(1).astype(str) + '%')
```

```
is_attack distribution:

is_attack
Benign    74.7%
Attack    25.3%
Name: proportion, dtype: object
```

The target variable is_attack is moderately imbalanced: 74.7% of sessions are Benign and 25.3% are Attacks. This is a notable characteristic because, as mentioned in Week 2 lecture, a naive model that predicts every session as Benign would achieve 74.7% accuracy without learning anything useful. This phenomenon was explicitly identified as the reason why accuracy alone is insufficient for imbalanced classification and that is why recall for the Attack class is the primary evaluation metric in this project.

Data Quality Assessment

```
protocol
TCP     2532
UDP     1378
tcp      572
Udp      557
ICMP     541
NaN      420
Name: count, dtype: int64
```

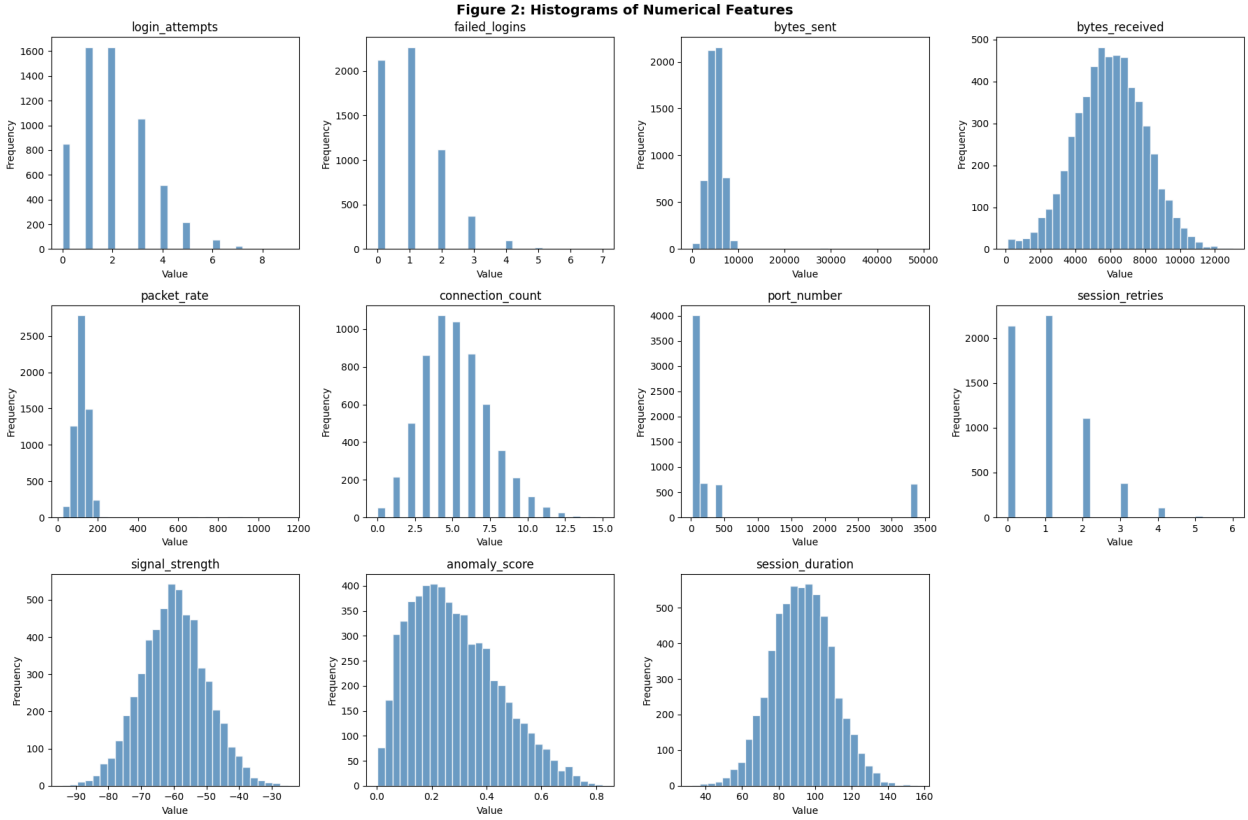
A data quality issue was found in protocol column. Referring to quality categories defined in Week 2 lecture, protocol inconsistency is a clear example of the ‘inconsistent data’ where same value is recorded using different cases, which may confuse the model and make it to treat (‘TCP’ and ‘tcp’) and (‘UDP’ and ‘Udp’) as different protocols.

```
# — Check missing values —
print('Missing values per column:')
missing = df.isnull().sum()
pct     = (missing / len(df) * 100).round(1)
print(pd.DataFrame({'Count': missing, '%': pct}))
```

```
Missing values per column:
                 Count    %
session_id           0  0.0
protocol           420  7.0
device_type          0  0.0
vpn_status           0  0.0
region               0  0.0
login_attempts       0  0.0
failed_logins        0  0.0
bytes_sent           0  0.0
bytes_received     420  7.0
packet_rate          0  0.0
connection_count     0  0.0
tcp_flags            0  0.0
port_number          0  0.0
session_retries      0  0.0
signal_strength    420  7.0
privilege_level      0  0.0
anomaly_score        0  0.0
session_duration     0  0.0
is_attack            0  0.0
```

The missing value rate of 7% per column is included in between the <5% threshold (where row deletion is appropriate) and the >40% threshold (where column deletion is appropriate). As mentioned in Week 3 lecture, the recommended approach is univariate imputation. The most frequent value was used for the categorical protocol column, and the median was used for numerical columns (bytes_received, signal_strength) because the median is less sensitive to outliers than the mean.

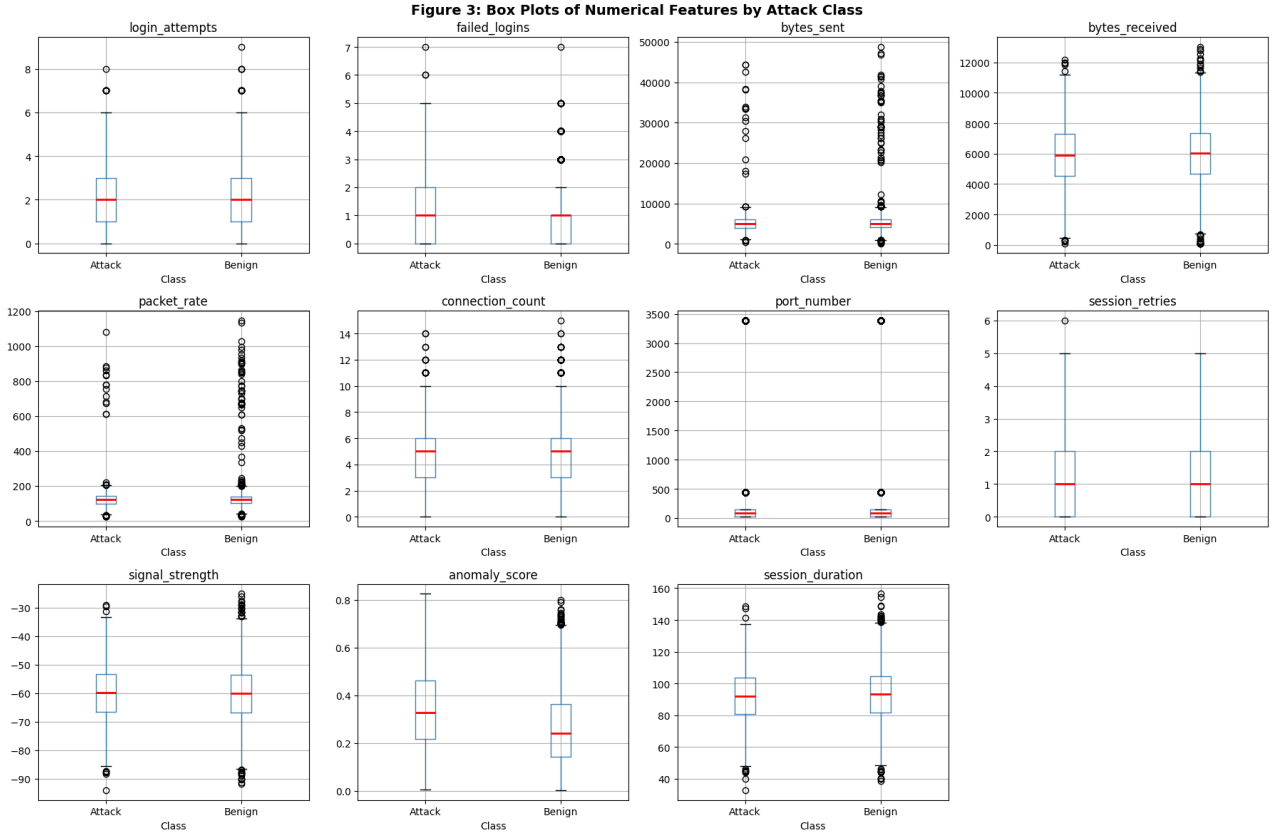
Visualisation of Histogram - Analysis



Visualising data distribution with histograms, indicate how often each value in the data set appears. Also, we can observe that authentication related attributes are more related to discrete right-skewed by containing more values near to the maximum of distribution. These observations confirm that the dataset contains important behavioural variation suitable for classification modelling and supports the inclusion of these variables as prediction in intrusion detection analysis.

Box Plot Visualisation\Analys

Figure 3: Box Plots of Numerical Features by Attack Class



For numeric attributes a box plots were used to show how values grouped by is_attack class revealed that anomaly_score shows a clear no overlap between Attack and Bening sessions, with Attack having an evident higher median anomaly score (0.326 vs 0.241). This bivariate analysis is explained in Week 2 lectures as “small or no overlap indicates an attribute that may have predictive power”.

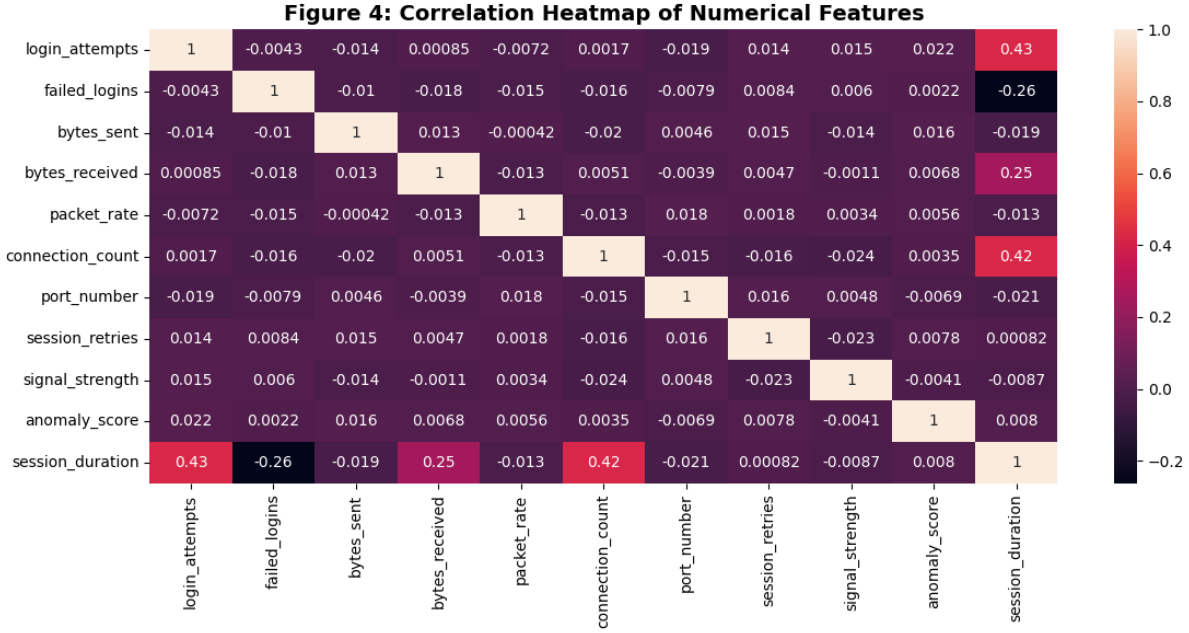
The failed_logins feature also shows some separation (mean 1.17 for Attacks vs 0.97 for Benign), though with high overlap, suggesting it has less predictive power on its own.

Features like session_duration show almost no separation between classes.

No negative counts or anomaly score was found outside physically plausible ranges, so no values required correction as errors. High values in bytes_sent represent heavy traffic rather than data entry mistakes.

Visualisation of Heatmap - Correlation Analysis

Figure 4: Correlation Heatmap of Numerical Features



The correlation heatmap of numerical features shows that most of them are considerably independent, suggesting low redundancy between predictions which is favourable for Naive Bayes. The strongest correlation observed was between session_duration and login_attempts (r=0.43) indicating that sessions involving more login attempts tend to last longer, this may reflect repeated login activity or suspicious login behaviour indicating credential-testing attempts.

Second strongest corelation exists between connection_count and Session_duration (r=0.42), suggesting that sessions with longer duration may involve a higher number of connections. This corelation could indicate persistent scanning or repeated connections attempts.

A third corelation exists between bytes_received and session_duration suggesting that longer sessions are associated with increased data transfer.

Methodology/Data Preparation

As identified in Week 3 lecture, data preparation follows four major tasks: Data cleaning, Data integration, Data transformation, and Data reduction.

Data Selection — Removing Useless Attributes

```
# — 3.1 Copy & Drop Irrelevant Feature —
data = df.copy()

# session_id is a unique identifier – not a predictive feature
data.drop(columns=['session_id'], inplace=True)
```

Data selection involves choosing what data from the data set is worth to be included in the analysis, for that session_id was dropped before any analysis. As mentioned in the Week 2 lecture on nominal variables, attributes that represent unique identifiers have the lowest information content and should be removed during the modelling phase. session_id won't influence the final outcome with its unique values.

Data Transformation - Inconsistent Categorical Values

```
# — 3.2 Fix Inconsistent Protocol Values —
# Standardise to uppercase to unify 'tcp'/'TCP', 'Udp'/'UDP'
data['protocol'] = data['protocol'].str.upper()
print('Protocol values after normalisation:')
print(data['protocol'].value_counts(dropna=False))
```

```
Protocol values after normalisation:
protocol
TCP     3104
UDP     1935
ICMP     541
NaN      420
Name: count, dtype: int64
```

For data transformation process the protocol column was standardised to uppercase to resolve the inconsistency identified during the data quality check. This resolves the inconsistent data problem that may confuse the model and make it to learn six different protocols instead of four.

Handling Missing Values

```
# — 3.3 Handle Missing Values —
cat_imputer = SimpleImputer(strategy='most_frequent')
data['protocol'] = cat_imputer.fit_transform(data[['protocol']]).ravel()

num_imputer = SimpleImputer(strategy='median')
data[['bytes_received', 'signal_strength']] = num_imputer.fit_transform(
    data[['bytes_received', 'signal_strength']]
)

print('Missing values after imputation:')
print(data.isnull().sum())
```

```
Missing values after imputation:
protocol            0
device_type         0
vpn_status          0
region              0
login_attempts      0
failed_logins       0
bytes_sent          0
bytes_received      0
packet_rate         0
connection_count    0
tcp_flags           0
port_number         0
session_retries     0
signal_strength     0
privilege_level     0
anomaly_score       0
session_duration    0
is_attack           0
dtype: int64
```

Second, missing values were handled using univariate imputation as described in Week 3. The protocol column received mode imputation (most frequent value = 'TCP'), while bytes_received and signal_strength received median imputation. Median was chosen over mean because bytes_received is right-skewed, and the median keeps the central tendency without being influenced by extreme values. This is highlighted in Week 3 lecture: 'use the median for data that contains outliers or skewed data'.

Feature Engineering — Creating New Attributes

```
# — 3.4 Feature Engineering —
# Feature 1: bytes_ratio
data['bytes_ratio'] = data['bytes_sent'] / (data['bytes_received'] + 1)

# Feature 2: login_failure_rate
data['login_failure_rate'] = data['failed_logins'] / (data['login_attempts'] + 1)

print('New features added: bytes_ratio, login_failure_rate')
data[['bytes_ratio', 'login_failure_rate']].describe()
```

```
New features added: bytes_ratio, login_failure_rate
```

	bytes_ratio	login_failure_rate
count	6000.000000	6000.000000
mean	1.138640	0.446977
std	2.374157	0.570549
min	0.027900	0.000000
25%	0.625924	0.000000
50%	0.843555	0.333333
75%	1.140435	0.666667
max	64.883211	6.000000

- bytes_ratio = bytes_sent / (bytes_received + 1). New feature created captures the relative weight of outbound versus inbound traffic. An attacker exfiltrating data would send more bytes than they receive, producing a high ratio. This technique is a ratio transformation that reveals patterns, not very evident from the actual columns.
- login_failure_rate = failed_logins / (login_attempts + 1). This feature captures the login attempts that failed. A brute-force attack generates many failed logins comparing to successful ones, making this a strong behavioural indicator.

Encoding Categorical Features

```
# ── 3.5 Encode Categorical Features ──
label_cols = ['protocol', 'device_type', 'vpn_status', 'region', 'tcp_flags', 'privilege_level']

le = LabelEncoder()
for col in label_cols:
    data[col] = le.fit_transform(data[col].astype(str))

# Encode target: Attack=1, Benign=0
data['is_attack'] = le.fit_transform(data['is_attack'])
print('Unique is_attack values after encoding:', data['is_attack'].unique())
print('Encoding: Attack=1, Benign=0')
data[label_cols].head()
```

```
Unique is_attack values after encoding: [1 0]
Encoding: Attack=1, Benign=0
```

	protocol	device_type	vpn_status	region	tcp_flags	privilege_level
0	1	0	1	0	2	1
1	0	2	0	1	3	1
2	1	2	0	0	2	2
3	0	2	1	3	3	2
4	2	3	0	2	1	2

All six categorical columns (protocol, device_type, vpn_status, region, tcp_flags, privilege_level) were encoded using LabelEncoder to integer digits making the data compatible with learning classifiers. The is_attack target was encoded as Attack=0, Benign=1.

Train/Test Split

```
# ── 3.7 Train / Test Split (70/30, stratified) ──
X_cls_train, X_cls_test, y_cls_train, y_cls_test = train_test_split(
    X_cls, y_cls, test_size=0.30, random_state=42, stratify=y_cls
)

X_reg_train, X_reg_test, y_reg_train, y_reg_test = train_test_split(
    X_reg, y_reg, test_size=0.30, random_state=42
)

print(f'Classification — Train: {X_cls_train.shape[0]}, Test: {X_cls_test.shape[0]}')
print(f'Regression     — Train: {X_reg_train.shape[0]}, Test: {X_reg_test.shape[0]}')
```

```
Classification — Train: 4200, Test: 1800
Regression     — Train: 4200, Test: 1800
```

The dataset was split into 70% training (4,200 rows) and 30% testing (1,800 rows) using a stratified random split (random_state=42). Stratification ensures that both the training and test sets maintain the 74.7% - 25.3% class ratio, preventing the test set from being accidentally more or less balanced than the full dataset.

Feature Scaling

```
# ── 3.8 Feature Scaling (for KNN / Regression) ──
scaler = StandardScaler()
X_cls_train_sc = scaler.fit_transform(X_cls_train)
X_cls_test_sc  = scaler.transform(X_cls_test)

X_reg_train_sc = scaler.fit_transform(X_reg_train)
X_reg_test_sc  = scaler.transform(X_reg_test)

from sklearn.preprocessing import MinMaxScaler
minmax_scaler_nb = MinMaxScaler()
X_cls_train_nb = minmax_scaler_nb.fit_transform(X_cls_train)
X_cls_test_nb  = minmax_scaler_nb.transform(X_cls_test)

print('Scaling applied. Data is now ready for all models.')
```

```
Scaling applied. Data is now ready for all models.
```

MinMaxNormalisation(Scaler) was applied to create non-negative features for ComplementNB. As mentioned in Week 5 lecture, variables are transformed to a range [0,1]. Decision Tree models do not require scaling because they make splits based on feature value thresholds rather than distances, so unscaled features were used for all tree-based models.

```
# Convert scaled classification data back to DataFrame
X_cls_train_norm_df = pd.DataFrame(
    X_cls_train_nb,
    columns=X_cls.columns
)

print("Before normalisation (min/max):")
display(X_cls_train.describe().loc[['min', 'max']])

print("After Min-Max normalisation (min/max):")
display(X_cls_train_norm_df.describe().loc[['min', 'max']])
```

Before normalisation (min/max):

	protocol	device_type	vpn_status	region	login_attempts	failed_logins	bytes_sent	bytes_received	packet_rate	connection_count	tcp_flags	port_number	session_retries	signal_strength	privilege_level	anomaly_score	bytes_ratio	login_failure_rate
min	0.0	0.0	0.0	0.0	0.0	0.0	100.000000	100.000000	25.113174	0.0	0.0	21.0	0.0	-94.085793	0.0	0.002210	0.027900	0.0
max	2.0	3.0	1.0	3.0	9.0	7.0	48743.941649	13019.206996	1147.060032	15.0	3.0	3389.0	6.0	-25.139054	2.0	0.824099	57.183719	6.0

After Min-Max normalisation (min/max):

	protocol	device_type	vpn_status	region	login_attempts	failed_logins	bytes_sent	bytes_received	packet_rate	connection_count	tcp_flags	port_number	session_retries	signal_strength	privilege_level	anomaly_score	bytes_ratio	login_failure_rate
min	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
max	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0

Modelling

Classification — Complement Naïve Bayes

Baseline Model

```
=== Baseline Naïve Bayes (alpha=0, norm=False) ===
              precision    recall  f1-score   support

      Benign       0.32      0.56      0.41       456
      Attack       0.80      0.59      0.68      1344

    accuracy                           0.59      1800
   macro avg       0.56      0.58      0.54      1800
weighted avg       0.68      0.59      0.61      1800
```

The baseline ComplementNB model was created using the manual settings (alpha=0, force_alpha=True, norm=False), exactly as specified in the lab 7 instructions.

```
# ── Train vs Test Accuracy — Baseline NB ──
train_pred_nb_bl = nb_baseline.predict(X_cls_train_nb)
print(f'Baseline NB | Train Acc: {accuracy_score(y_cls_train, train_pred_nb_bl):.4f}  '
      f'Test Acc: {accuracy_score(y_cls_test, y_pred_nb_baseline):.4f}')
```

```
Baseline NB | Train Acc: 0.5983  Test Acc: 0.5856
```

The Train Accuracy of 0.5983 and Test Accuracy of 0.5856 suggests the model is not overfitting, just very small underfitting, because NB assumes feature independence and can not capture complex features related to network activity.

Improved Model

```
# — 4A.3 GridSearchCV — 5-Fold —
param_grid_nb = {
    'alpha': [0.01, 0.1, 0.5, 1.0, 2.0, 10.0],
    'norm':  [True, False]
}

gs_nb_5 = GridSearchCV(
    ComplementNB(), param_grid_nb, cv=5,
    scoring='accuracy', return_train_score=True, n_jobs=-1
)
gs_nb_5.fit(X_cls_train_nb, y_cls_train)

print('Best params (5-fold):', gs_nb_5.best_params_)
print(f'Best CV accuracy:     {gs_nb_5.best_score_:.4f}')
```

```
Best params (5-fold): {'alpha': 10.0, 'norm': True}
Best CV accuracy:     0.7460
```

```
# — 4A.4 GridSearchCV — Stratified 10-Fold —
gs_nb_10 = GridSearchCV(
    estimator=ComplementNB(),
    param_grid=param_grid_nb,
    cv=StratifiedKFold(10),
    scoring='accuracy',
    n_jobs=-1
)
gs_nb_10.fit(X_cls_train_nb, y_cls_train)

print('Best params (Stratified 10-fold):', gs_nb_10.best_params_)
print(f'Best CV accuracy:                {gs_nb_10.best_score_:.4f}')
```

```
Best params (Stratified 10-fold): {'alpha': 10.0, 'norm': True}
Best CV accuracy:                 0.7460
```

```
=== Best Naïve Bayes {'alpha': 10.0, 'norm': True} ===

              precision    recall  f1-score   support

      Benign       0.39      0.03      0.06       456
      Attack       0.75      0.98      0.85      1344

    accuracy                           0.74      1800
   macro avg       0.57      0.51      0.45      1800
weighted avg       0.66      0.74      0.65      1800
```

Hyperparameter tuning was performed using GridSearchCV with two validation strategies: 5-fold cross-validation and Stratified 10-fold cross-validation to search for the best parameters:

- alpha: [0.01, 0.1, 0.5, 1.0, 2.0, 10.0] - Testing different smoothing strengths
- norm: [True, False] - Testing if weight normalization helps

The Stratified 10-fold search identified the best parameters as alpha=10.0 reflecting strong smoothing, norm=True sets normalise complement log probabilities, which help when features have unequal scales., with a cross-validation accuracy of 0.7460. On the held-out test set, this model achieved 74% accuracy with an attack recall of 0.98, correctly identifying 98% of all attack sessions.

Confusion Matrix

Figure 7: Baseline Naïve Bayes — Confusion Matrix

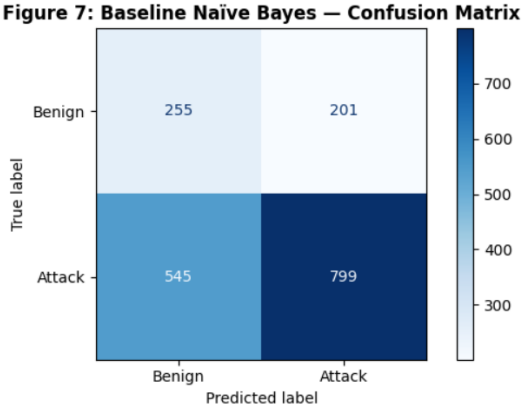
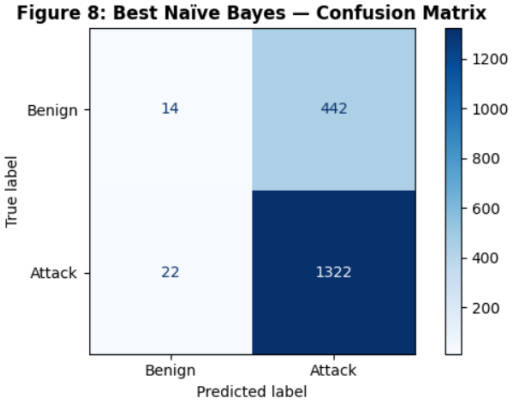


Figure 8: Best Naïve Bayes — Confusion Matrix



Comparing the baseline Naïve Bayse model, the improved one shows significant attack detection increasing by correctly identifying 1322 attacks instances and reducing missed attacks from 545 to just 22. Despite this improvement, a large increase in false positive can be observed as attack rising from 201 to 442. Overall, the model prioritises detecting benign attacks but sacrifices the accuracy.

Classification — Decision Tree

Baseline Model

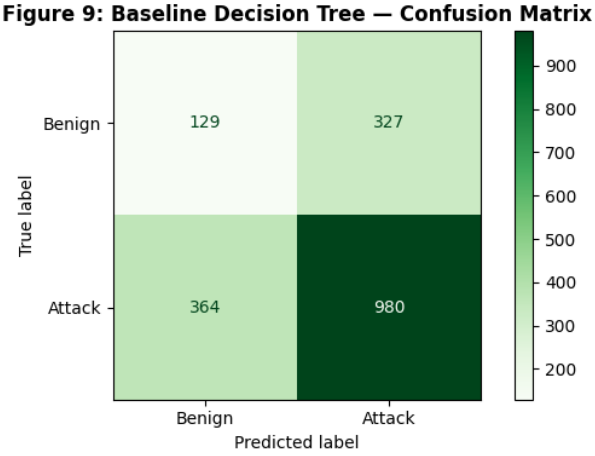
```
=== Baseline Decision Tree ===
              precision    recall  f1-score   support

      Benign       0.26      0.28      0.27       456
      Attack       0.75      0.73      0.74      1344

    accuracy                           0.62      1800
   macro avg       0.51      0.51      0.51      1800
weighted avg       0.63      0.62      0.62      1800

Train Acc: 1.0000  Test Acc:  0.6161
```

Figure 9: Baseline Decision Tree — Confusion Matrix



A DecisionTreeClassifier was trained as the baseline. The training accuracy was 1.0000, perfect on the training data, while test accuracy was only 62%. As described in Week 4 lecture, this is a case of overfitting as a result of attempting to accommodate unusual cases in the data. The confusion Matrix shows strong detection of attack instances 980, but a quite high number of misclassifications, including 364 missed attacks and 327 benign incorrectly labelled as attacks. Overall, the model has a moderate effectiveness in identifying malicious activity but limited reliability for distinguishing benign traffic due to poor generalisation performance.

Improved Model

```
Best params (DT): {'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2}
Best CV accuracy: 0.7524
```

To address overfitting, the Decision Tree was tuned using GridSearchCV:

- max_depth: [3, 5, 10, None] — limiting depth prevents the tree from memorising training noise
- min_samples_split: [2, 5, 10] — minimum samples required to split a node
- criterion: ['gini', 'entropy'] — the impurity measure used for splitting

The best parameters found were criterion=gini, max_depth=3, min_samples_split=2, with a 10-fold CV accuracy of 0.7524.

```
=== Best Decision Tree {'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 2} ===
              precision    recall  f1-score   support

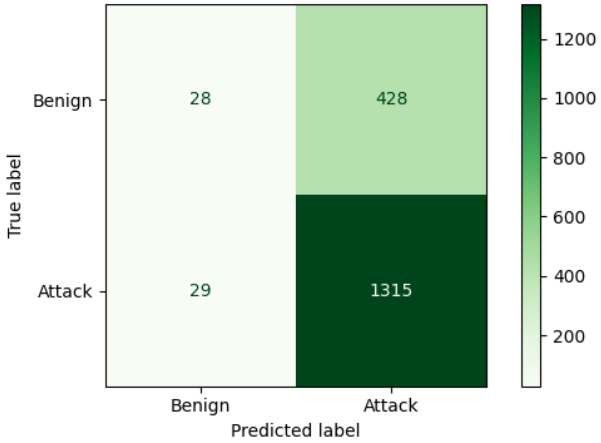
      Benign       0.49      0.06      0.11       456
      Attack       0.75      0.98      0.85      1344

    accuracy                           0.75      1800
   macro avg       0.62      0.52      0.48      1800
weighted avg       0.69      0.75      0.66      1800

Train Acc: 0.7550  Test Acc:  0.7461
```

After improving the model, it went from 100%/62% (train/test) to approximately 75%/75%, indicating the overfitting has been resolved.

Figure 10: Best Decision Tree — Confusion Matrix



Improved Decision Tree shows a strong improvement in attack detection, correctly identifying 1315 attacks and reducing missed attacks from 364 to just 29. However, it still misclassifies a large number of benign sessions (428) as attacks, indicating a higher false positive rate.

Regression — Linear Regression and Decision Tree Regressor

Linear Regression

```
=== Baseline Linear Regression ===
MAE:  9.7793
RMSE: 12.3144
R²:   0.4752
```

The model was trained to predict session_duration using regression features and achieved MAE of 9.78, RMSE of 12.31, and R^2 of 0.4752. An R^2 of 0.48 indicates moderate predictive performance in estimating session duration.

Decision Tree Regressor

```
=== Baseline Decision Tree Regressor ===
MAE:  14.9395
RMSE: 18.6265
R²:   -0.2007
```

Because of overfiting to the data trainignand and poor generalisation, the model prediction is very bad performed with a higher MAE of 14.9395, RMSE of 18.6265, and a negative R^2 value of −0.2007.

```
Best DT Regressor params: {'max_depth': 5, 'min_samples_leaf': 1, 'min_samples_split': 2}
MAE:  10.6479
RMSE: 13.4448
R²:   0.3744
```

After improving the model with GridSearchCV the best parameters were max_depth=5, min_samples_leaf=1, min_samples_split=2, achieving R^2 = 0.3744, lower than Linear Regression.

Evaluation & Results

Classification Model Comparison

=== Classification Model Comparison ===

Model	Accuracy	Precision	Recall	F1-Score
Baseline Naïve Bayes	0.5856	0.7990	0.5945	0.6817
Best Naïve Bayes (GridSearchCV)	0.7422	0.7494	0.9836	0.8507
Baseline Decision Tree	0.6161	0.7498	0.7292	0.7393
Best Decision Tree (GridSearchCV)	0.7461	0.7544	0.9784	0.8520

Both tuned models (Best NB and Best DT) achieved the same test accuracy of 75% and identical attack recall of 0.98. This shows that two completely different models come up on the same prediction results after optimisation.

In cybersecurity terms, a recall of 0.98 means the model misses only 2% of attacks. However, many normal sessions are incorrectly categorised as attack, making the precision for Benign attacks remains low (39% - 49%).

Feature Importance

The best Decision Tree assigned the following feature importances:

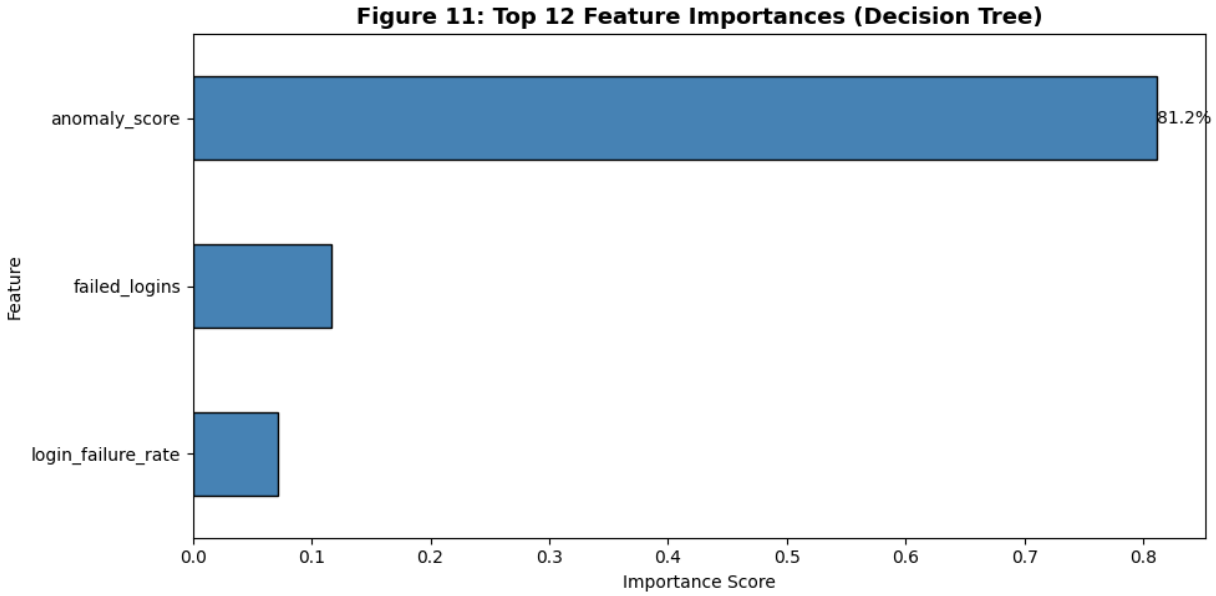


Figure 11: Top 12 Feature Importances (Decision Tree)

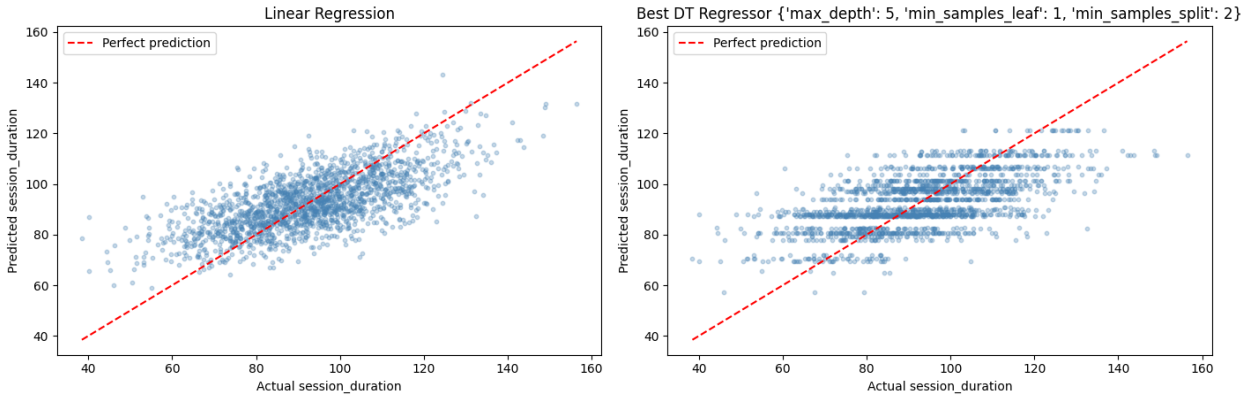
The anomaly_score is the most important predictor, suggesting that the Decision Tree depends a lot on this variable. The engineered feature login_failure_rate can be observed among the top predictors variables, confirming that was useful, as it captures additional information about suspicious login behaviour beyond the original variables alone.

Regression Model Comparison

```
=== Regression Model Comparison ===
```

Model	MAE	RMSE	R^2
Linear Regression	9.7793	12.3144	0.4752
Baseline DT Regressor	14.9395	18.6265	-0.2007
Best DT Regressor (GridSearchCV)	10.6479	13.4448	0.3744

Figure 12: Actual vs Predicted — Regression Models



The graphs indicate that the Linear Regression model outperforms the Decision Tree Regressor, indicating better overall accuracy and a stronger ability to capture the relationship between features and session duration.

The R^2 of 0.475 means the model explains half of the variation in session duration.

The fact that both regression models have modest R^2 values is informative that session duration is not well predicted by network traffic features alone.

Conclusions and Reflection on Learning

Summary of Findings

This project applied the full CRISP-DM lifecycle to a synthetic network security dataset. The analysis revealed that anomaly_score is the most important predictor of attack sessions, contributing over 81% of the Decision Tree's classification decisions. Two new engineered features (bytes_ratio and login_failure_rate) were created and validated, with login_failure_rate appearing as the third most important feature.

Both model after tuning achieved 75% accuracy and 98% attack recall. The high recall is particularly the most important in the cybersecurity domain showing that the model catches 98 out of every 100 attacks.

The regression analysis showed that session duration is moderately predictable from network features ($R^2 \sim 0.48$). So, Linear Regression outperformed the Decision Tree Regressor, suggesting an approximately linear relationship between the features and session duration.

The class imbalance (74.7% Benign) was handled through splitting and by prioritising recall, but further techniques such as SMOTE oversampling or class-weighted loss functions could be explored to improve precision for Benign sessions, reducing the false alarm rate that security analysts would need to manage.

Reflection on Learning

This project demonstrated that the data understanding and preparation phases are not setup steps, they are where the most important analytical decisions are made. Identifying the protocol inconsistency, choosing median over mean for skewed columns, and engineering the login_failure_rate feature each required applying concepts from the lectures directly to real dataset. The improvement from the baseline Decision Tree model (62% test accuracy and 100% training accuracy) to the tuned version (75% test accuracy) highlighted how important model adjustment is for improving performance and reducing overfitting.

The observation that both Naïve Bayes and Decision Tree models achieved similar results also demonstrates an important understanding when a dataset includes a highly predictive feature such as anomaly_score. This suggests that model performance often depends more on the quality of the features than on the choice of algorithm itself.

References

Project_Script.ypynb

Week 2: Data Exploration and Understanding. COMP H2706 Lecture Slides.

Week 3: EDA II. COMP H2706 Lecture Slides.

Week 4: Classification Model. COMP H2706 Lecture Slides.

Week 5: Data Preparation-1. COMP H2706 Lecture Slides.

Week 12: Regression. COMP H2706 Lecture Slides.

Chapman, P. et al. (1999). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc. Available at: https://www.datascience-pm.com/crisp-dm-2/

Rennie, J. D. M. et al. (2003). Tackling the Poor Assumptions of Naive Bayes Text Classifiers. Proceedings of the 20th International Conference on Machine Learning (ICML-2003).